DevOps Lab Program- 8

Name: Aditi Narasimhan
USN: 1RV24MC005

Project Structure:

```
1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-8$ tree
.
├── nodejs-configmap.yaml
├── nodejs-pod.yaml
└── server.js

1 directory, 3 files
```

Steps for execution:

Step 1: Run *minikube start* to start minikube

```
1rv24mc005_aditi@aditi-Inspiron-3584:~$ minikube start
😄  minikube v1.37.0 on Ubuntu 24.04
✨  Using the docker driver based on existing profile
👍  Starting "minikube" primary control-plane node in "minikube" cluster
🚜  Pulling base image v0.0.48 ...
🔄  Restarting existing docker container for "minikube" ...

🧯  Docker is nearly out of disk space, which may cause deployments to fail! (87% of capacity). You can pass '--for
ce' to skip this check.
💡  Suggestion:

    Try one or more of the following to free up space on the device:

    1. Run "docker system prune" to remove unused Docker data (optionally with "-a")
    2. Increase the storage allocated to Docker for Desktop by clicking on:
    Docker icon > Preferences > Resources > Disk Image Size
    3. Run "minikube ssh -- docker system prune" if using the Docker container runtime
🍿  Related issue: https://github.com/kubernetes/minikube/issues/9024

🐳  Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
🔎  Verifying Kubernetes components...
    ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟  Enabled addons: storage-provisioner, default-storageclass

❗  /usr/bin/kubectl is version 1.28.15, which may have incompatibilities with Kubernetes 1.34.0.
    ▪ Want kubectl v1.34.0? Try 'minikube kubectl -- get pods -A'
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

Step 2: Check the status of minikube using *minikube status*

```
1rv24mc005_aditi@aditi-Inspiron-3584:~$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

Step 3: Write a Node.js app in a file called server.js as follows:

```
1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-8$ cat server.js
const http = require("http");
const port = 3000;

const server = http.createServer((req,res) => {
res.end("Hello from Node.js running inside Kubernetes!");
});

server.listen(port,() => {
console.log(`Server running at port $(port)`);
});
```

Step 4:
Create a ConfigMap file called nodejs-configmap.yaml with the following contents:

```
1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-8$ cat nodejs-configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: nodejs-app-config
data:
  server.js: |
    const http = require("http");
    const port = 3000;
    const server = http.createServer((req,res) => {
      res.end("Hello from Node.js running inside Kubernetes!");
    });

    server.listen(port, () => {
      console.log(`Server running at $(port)`);
    });
```

Step 5:
Create a file for the pod using Node.js as a base image:

```
1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-8$ cat nodejs-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: program-8
  labels:
    app: nodejs
spec:
  containers:
    - name: nodejs
      image: node:18
      command: ["node", "/usr/src/app/server.js"]
      ports:
        - containerPort: 3000
      volumeMounts:
        - name: app-code
          mountPath: /usr/src/app
  volumes:
    - name: app-code
      configMap:
        name: nodejs-app-config
```

Step 6:
Apply the YAML files and verify by checking pod status:

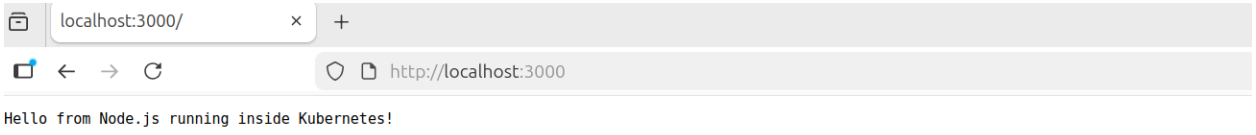
```
1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-8$ kubectl apply -f nodejs-configmap.yaml
configmap/nodejs-app-config created
1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-8$ kubectl apply -f nodejs-pod.yaml
pod/program-8 created
1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-8$ kubectl get pods
NAME        READY   STATUS              RESTARTS   AGE
program-8   0/1     ContainerCreating   0          7s

1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-8$ kubectl get pods
NAME        READY   STATUS    RESTARTS   AGE
program-8   1/1     Running   0          6m26s
```

Step 7:
Access the web app by forwarding the port via kubectl:
kubectl port-forward pod/program-8 3000:3000

```
1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-8$ kubectl port-forward pod/program-8 3000:3000
Forwarding from 127.0.0.1:3000 -> 3000
Forwarding from [::1]:3000 -> 3000
```

Output:



Step 7:
Delete all pods, services and deployments:

```
^C1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-8$ kubectl delete pods --all
pod "program-8" deleted
1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-8$ kubectl delete services --all
service "kubernetes" deleted
```